

*Universidad San Carlos de Guatemala
Centro Universitario de Occidente
Ingeniería en Ciencias y Sistemas
Laboratorio de Sistemas de Bases de
datos 1
ING. Otto Soto*



Practica No.1

*Jueves, 03 de septiembre, 2026
Quetzaltenango*

*Juan Carlos Rodríguez López
202231970*

Problema 1

El sistema le pide al usuario si desea probar la conectividad antes, si no se le pide que ingrese su numero de tarjeta.

```
total de camisas despachadas: 52
carlos@carlos-HP-ENVY-x360-2-in-1-Laptop-15-ew1xxx:~/Documentos$
=====
SISTEMA DE PAGO EN LÍNEA
=====
¿Desea realizar verificación de estado? (1 = Sí, 0 = No): 0
```

El sistema le pide al usuario que ingrese una palabra para probar el canal.

```
carlos@carlos-HP-ENVY-x360-2-in-1-Laptop-15-ew1xxx:~/Documentos$
Ingrese una palabra para probar el canal: que hay
```

El sistema le indica al usuario que el canal esta activo y le da la cadena invertida, y le pide que ingrese su numero de tarjeta.

```
[HIJO] Canal activo. Confirmación (cadena invertida): yah euq
Ingrese su número de tarjeta (entre 1000 y 9999):
```

El usuario ingresa un número impar, el sistema rechazara el pago

```
Ingrese su número de tarjeta (entre 1000 y 9999): 1005
[PADRE] Respuesta del sistema de pago: PAGO_RECHAZADO
```

El usuario ingresa un número par, el sistema rechazara el pago

```
Ingrese su número de tarjeta (entre 1000 y 9999): 1006
[PADRE] Respuesta del sistema de pago: PAGO_APROBADO
```

Preguntas

- **¿Qué ocurriría si el hijo intentara leer de la tubería antes de que el padre haya escrito algo?**

La llamada a sistema `read()` sobre un pipe en modo bloqueante (comportamiento por defecto) suspende la ejecución del proceso. Si el hijo intenta leer antes de que el padre escriba, el proceso hijo se mantendrá bloqueado en estado de espera hasta que el padre coloque datos en el búfer de la tubería.

- **La segunda fase requiere que el hijo envíe una respuesta de vuelta al padre.
¿Por qué no es suficiente con la misma tubería usada en la verificación?**

Las tuberías anónimas son unidireccionales. Si ambos procesos leen y escriben sobre la misma tubería, surge una condición de carrera: el padre podría terminar leyendo el mensaje que él mismo acaba de escribir antes de que el hijo lo reciba, o el hijo podría consumir su propia respuesta, corrompiendo la comunicación. Se requieren dos tuberías separadas para garantizar un flujo bidireccional seguro (una para Padre $\rightarrow$ Hijo y otra para Hijo $\rightarrow$ Padre).

- **¿Qué problema concreto surgiría si ambos procesos intentaran leer y escribir sobre la misma tubería?**

El problema concreto es la aparición de una condición de carrera (race condition) por interferencia entre procesos, ya que el búfer del pipe no distingue qué proceso colocó los datos

Problema 2

Muestra la bienvenida del sistema del Problema 2 y el inicio de la solicitud para el ingreso de los 20 pedidos del turno.

```
carlos@carlos-HP-ENVY-x360-2-in-1-Laptop-15-ew1xxx
=====
      BODEGA DE DISTRIBUCIÓN DE CAMISAS
=====
Ingrese los 20 pedidos (valores entre 1 y 100):
Pedido # 1: 
```

Muestra la secuencia de ingreso de datos y la validación de límites. Se aprecia el control de errores al rechazar el valor 245 en el pedido #11 y solicitar un valor válido antes de continuar.

```
Ingrese los 20 pedidos (valores entre 1 y 100):
Pedido # 1: 5
Pedido # 2: 17
Pedido # 3: 46
Pedido # 4: 36
Pedido # 5: 84
Pedido # 6: 25
Pedido # 7: 92
Pedido # 8: 34
Pedido # 9: 24
Pedido #10: 23
Pedido #11: 245
    [!] Valor inválido. Debe ser entre 1 y 100.
Pedido #11: 23
Pedido #12: 12
Pedido #13: 13
Pedido #14: 78
Pedido #15: 97
Pedido #16: 56
Pedido #17: 45
Pedido #18: 36
Pedido #19: 63
Pedido #20: 73
```

Muestra el momento en que el proceso Productor (Padre) concluye la lectura de los 20 pedidos desde la consola y escribe los datos en la tubería anónima antes de cerrar su extremo de escritura.

```
[PADRE] Registro completo. Colocando pedidos en la banda transportadora...
```

Evidencia la ejecución de los dos procesos Hijos independientes (Estación 1 con PID 3210724 y Estación 2 con PID 3210725) al detectar el cierre del pipe (EOF) e iniciar la emisión de sus reportes.

```
=====
=====
REPORTE FINAL: ESTACIÓN 2 (PID: 3210725)
REPORTE FINAL: ESTACIÓN 1 (PID: 3210724)
=====
=====
```

Muestra el resultado de la competencia por la tubería compartida (race condition), donde cada estación logró tomar 10 pedidos de la banda, despachando 504 y 378 camisas respectivamente.

```
Pedidos procesados: 10
Pedidos procesados: 10
Total de camisas despachadas: 504
Total de camisas despachadas: 378
```

Preguntas

- ***¿Es posible predecir de antemano cuántos pedidos procesará cada estación? Relaciona tu respuesta con el concepto de race condition.***

No es posible predecir la cantidad exacta de pedidos que tomará cada estación. Existe una condición de carrera (*race condition*) provocada por la competencia activa entre los dos procesos hijos sobre el descriptor de lectura del pipe compartido. El planificador de la CPU (*CPU Scheduler*) decide qué proceso despierta primero para ejecutar `read()` cada vez que hay un pedido disponible en la tubería.

- ***¿Qué pasaría si una de las estaciones no cerrará el extremo de escritura que heredó del coordinador?***

Al hacer `fork()`, los hijos heredan copias exactas de los descriptores del padre. Si una estación no cierra su copia del extremo de escritura (`pipe_banda[1]`), el contador de referencias de escritura en el kernel nunca llegará a cero. Como consecuencia, cuando la tubería quede vacía, la función `read()` no devolverá 0 (EOF) y los procesos se quedarán bloqueados indefinidamente esperando más datos.

Problema 3

Muestra la ejecución inicial de `centro.c`, creando la FIFO y quedando a la espera de reportes de sucursales independientes.

```
carlos@carlos-HP-ENVY-x360-2-in-1-Laptop-15-ew1xxx:~$ ./centro.c
=====
CENTRO DE OPERACIONES (SERVIDOR)
=====
[CENTRO] Esperando reportes de sucursales...
```

Muestra el programa `sucursal.c` abriendo la tubería con nombre en modo escritura desde otra terminal e ingresando el reporte del día (2000).

```
carlos@carlos-HP-ENVY-x360-2-in-1-Laptop-15-ew1xxx:~/Documentos/Sistemas-op$ ./sucursal.c
=====
SUCURSAL DE PAGOS
=====
Ingrese el monto total reportado (o 'cerrar' para finalizar el día): 2000
```

Evidencia cómo el proceso servidor recibe la información del FIFO, registra la hora exacta de recepción (18:28:24) y actualiza el total acumulado a Q2000.00.

```
[18:28:24] Reporte recibido: Q2000.00 | Total acumulado: Q2000.00
```

Muestra la confirmación enviada por el proceso cliente tras transmitir exitosamente la información por la tubería con nombre.

```
[SUCURSAL] Reporte enviado exitosamente al Centro de Operaciones.
```

Preguntas

- **¿Por qué en este caso no fue necesario usar `fork()` para relacionar `sucursal.c` con `centro.c`? Explica qué característica de las `FIFO` hace posible que dos programas completamente independientes se comuniquen.**

Las `FIFOs` no requieren parentesco por `fork()` porque existen físicamente en el sistema de archivos con una ruta asignada (`mkfifo`). Esta característica permite que cualquier proceso independiente con permisos adecuados pueda interactuar con la tubería abriéndola directamente mediante la llamada `open()` usando su ruta de archivo.

- **¿Qué ocurre si `sucursal.c` intenta abrir la `FIFO` antes de que `centro.c` la haya creado?**

La llamada a sistema `open()` fallará y devolverá `-1` estableciendo el error `ENOENT`. El proceso `sucursal.c` no podrá conectarse a la tubería y el programa se interrumpirá a menos que contemple una gestión de errores o cree la `FIFO` por sí mismo.